


# Software Engineering Practice

## Analysis and Design of a PoS System

Name: Yao Jiaqi

NUIST ID: 202383890030

SETU ID: 20109999

Email: 2271427187@qq.com

GitHub URL:

Demo Video URL:

11/ 20 /2026

# Contents

Part 1 – The Agile Process ..... 3

1.Project Vision, Scope, and Product Goal ..... 3

2.Scrum Role Mapping and Collaboration Notes ..... 3

3.Product Backlog ..... 4

4.Sprint Planning ..... 5

5.Burndown Charts and Sprint Review & Retrospective ..... 7

Part 2 – The point-of-sale (POS) system ..... 8

Introduction ..... 8

1 Use Case Model ..... 8

2 System Sequence Diagram (SSD) ..... 9

3 Domain Model ..... 10

4 Logical Architecture ..... 11

5 Object Design ..... 11

6 Implementation Summary ..... 13

# Part 1 – The Agile Process

## 1.Project Vision, Scope, and Product Goal

Vision: To build a lightweight, standalone POS (Point of Sale) system for a small supermarket. The system aims to help cashiers quickly process sales, handle cash payments, print receipts, and manage returns, while synchronizing inventory updates after each transaction to reduce manual bookkeeping errors.

In Scope:

Sales Processing: Create new sale, enter items, calculate running total, accept cash payment, generate receipt, update inventory.

Return Handling: Look up historical sales, record returned items, calculate refunds, update inventory.

Startup Initialization: Load product catalog.

Out of Scope: Credit card/mobile payments (cash only for this phase), loyalty points and promotional discounts, multi-store networking and remote inventory sync, complex reporting and data analysis.

Product Goal: Within 3 Sprints (9 days total), deliver a runnable standalone POS system that supports both cash sales and return main workflows, and verify that the main success scenarios can be executed end-to-end through a demo. Clearly defining the scope boundary reflects the Scrum Product Goal and prevents scope creep.

## 2.Scrum Role Mapping and Collaboration Notes

This is a single-person project. The three Scrum roles are held by the author, but the process still follows Scrum practices:

| **Scrum Role** | **Assigned To** | **Responsibilities** |
|---|---|---|
| Product Owner | Author | Write user stories, prioritize backlog, define acceptance criteria, conduct Sprint Reviews |
| Scrum Master | Author | Enforce timeboxing, facilitate retrospectives, remove blockers (e.g., tool issues) |
| Development Team | Author | Use case modeling, domain modeling, design, coding, testing |

# 3.Product Backlog

| ID | User Story | Priority | Points | Dependency |
|---|---|---|---|---|
| US1 | Cashier starts a new sale. | Must | 3 | None |
| US2 | Cashier enters item ID; system displays price and accumulates total. | Must | 5 | US1 |
| US3 | Cashier accepts cash payment and prints receipt. | Must | 5 | US1,US2 |
| US4 | Cashier looks up historical sales to process a return. | Should | 5 | US3 |
| US5 | Cashier records returned items and updates inventory. | Should | 3 | US4 |

Complete User Stories and Acceptance Criteria

- US1: As a cashier, I want to start a new sale so that I can enter items.

Acceptance Criteria: ① Clicking "New Sale" creates a Sale instance; ② The interface displays a total of 0.

- US2: As a cashier, I want to enter an item ID so that the system displays the description, price, and accumulates the total.

Acceptance Criteria: ① After entering a valid ID, the description and price appear within 1 second; ② The total accumulates correctly; ③ An error message is shown for invalid IDs.

- US3: As a cashier, I want to accept cash payment so that the sale is completed and a receipt is printed.

Acceptance Criteria: ① Enter the amount tendered, and the system calculates change; ② Generate a receipt containing item details, total, and change; ③ Inventory decreases by 1 accordingly.

- US4: As a cashier, I want to look up historical sales so that I can process a return.

Acceptance Criteria: ① Enter a sale ID to find the corresponding sale; ② Display the original sale's item details.

- US5: As a cashier, I want to record returned items so that the refund is calculated and inventory is updated.

Acceptance Criteria: ① After selecting returned items, the system calculates the refund amount; ② Inventory increases by 1 accordingly; ③ Generate a return receipt.

# 4.Sprint Planning

## 4.1 Sprint 1 Plan

Sprint Goal: Implement the main sales entry workflow — start a new sale, enter items, and calculate the running total.

Timebox: 3 days (Day 1 - Day 3)

Sprint Backlog:

| **Task** | **Related Story** | **Estimate (Hours)** |
|---|---|---|
| Draw use case diagram, write Process Sale use case text | US1, US2 | 2 |
| Draw Process Sale SSD | US1, US2 | 1 |
| Draw domain model class diagram | US1, US2 | 1.5 |
| Code ProductDescription, SalesLineItem, Sale, ProductCatalog | US2 | 3 |
| Code Register.enterItem() and makeNewSale() | US1, US2 | 2 |
| Console test: Enter two items and print total | US1, US2 | 1 |
| Merge sprint1 branch to main, tag v0.1 | — | 0.5 |

## 4.2 Sprint 2 Plan

Sprint Goal: Implement cash payment, change calculation, receipt generation, and inventory deduction.

Timebox: 3 days (Day 4 - Day 6)

Sprint Backlog:

| Task | Related Story | Estimate (Hours) |
|---|---|---|
| Draw Design Class Diagram (DCD) | US3 | 1.5 |
| Code CashPayment class | US3 | 1 |
| Code Sale.makePayment() and getBalance() | US3 | 2 |
| Code receipt printing method | US3 | 1.5 |
| Code Inventory deduction | US3 | 1 |
| Test: Complete sales flow including change | US3 | 1 |
| Merge sprint2 to main, tag v0.2 | —— | 0.5 |

Definition of Done: Complete sales flow runs end-to-end (entry → payment → change → receipt → inventory -1); sequence diagram and DCD are completed.

Risks & Mitigation: If receipt formatting takes longer than expected, simplify to plain text output and do not pursue layout formatting

## 4.3 Sprint 3 Plan

Sprint Goal: Implement the main return workflow — look up a sale, record returns, calculate refunds, and replenish inventory.

Timebox: 3 days (Day 7 - Day 9)

Sprint Backlog:

| Task | Related Story | Estimate (Hours) |
|---|---|---|
| Write Handle Returns use case text | US4, US5 | 1 |
| Draw return process sequence diagram | US4, US5 | 1.5 |
| Code historical sale lookup | US4 | 2 |
| Code return processing and refund calculation | US5 | 3 |
| Code inventory replenishment | US5 | 1 |
| Test: Full flow from sale to return | US4, US5 | 1.5 |
| Merge sprint3 to main, tag v1.0 | —— | 0.5 |

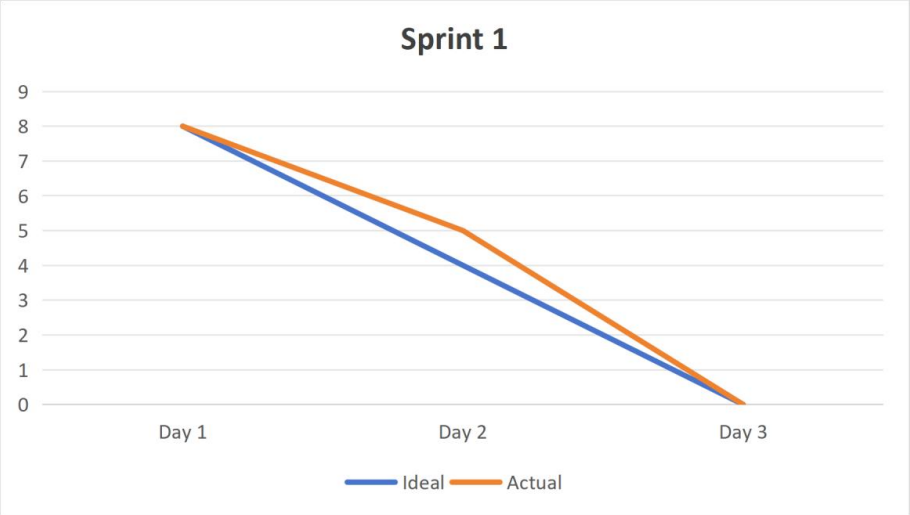
Definition of Done: Full flow from sale to return runs end-to-end; refund amount is

correct; inventory replenishment is correct; system reaches v1.0 demo-ready state.

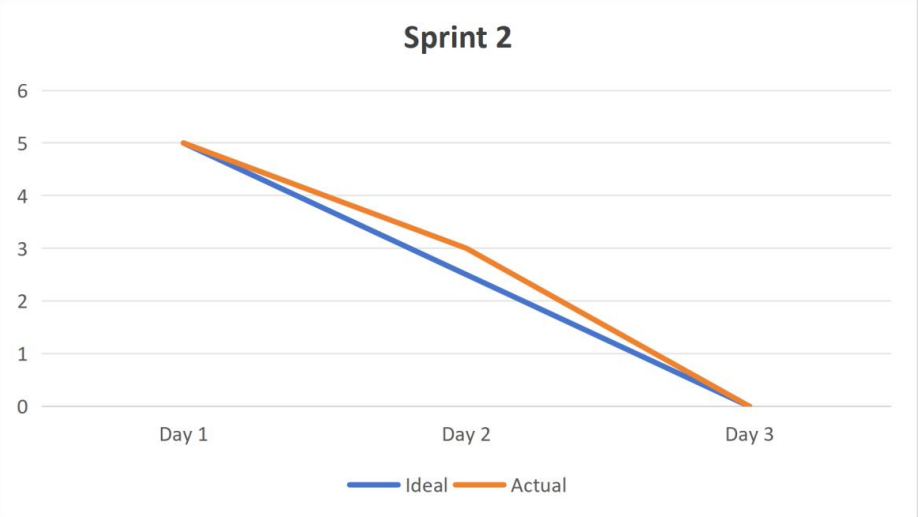
Risks & Mitigation: If persistence for historical sales lookup cannot be finished in time, simplify to in-memory List storage and note it as a simplified design in the report.

# 5.Burndown Charts and Sprint Review & Retrospective

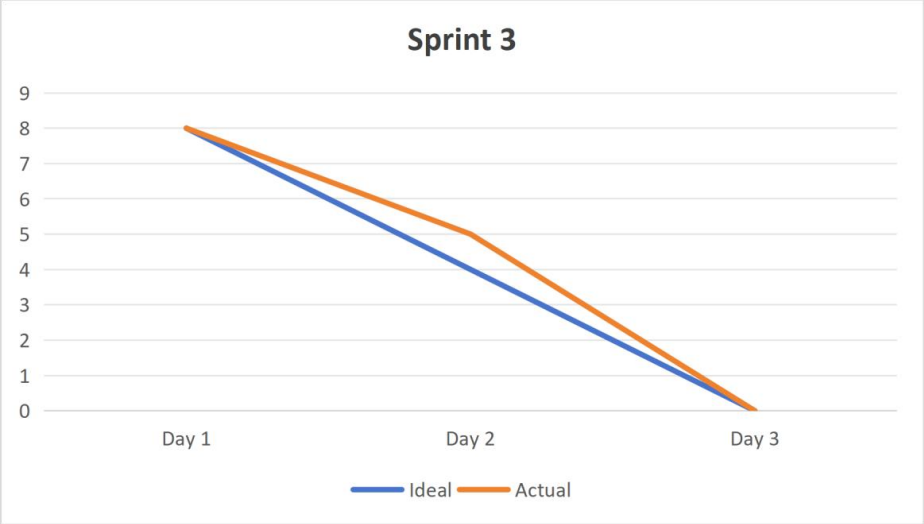
## Sprint 1




## Sprint 2




## Sprint 3



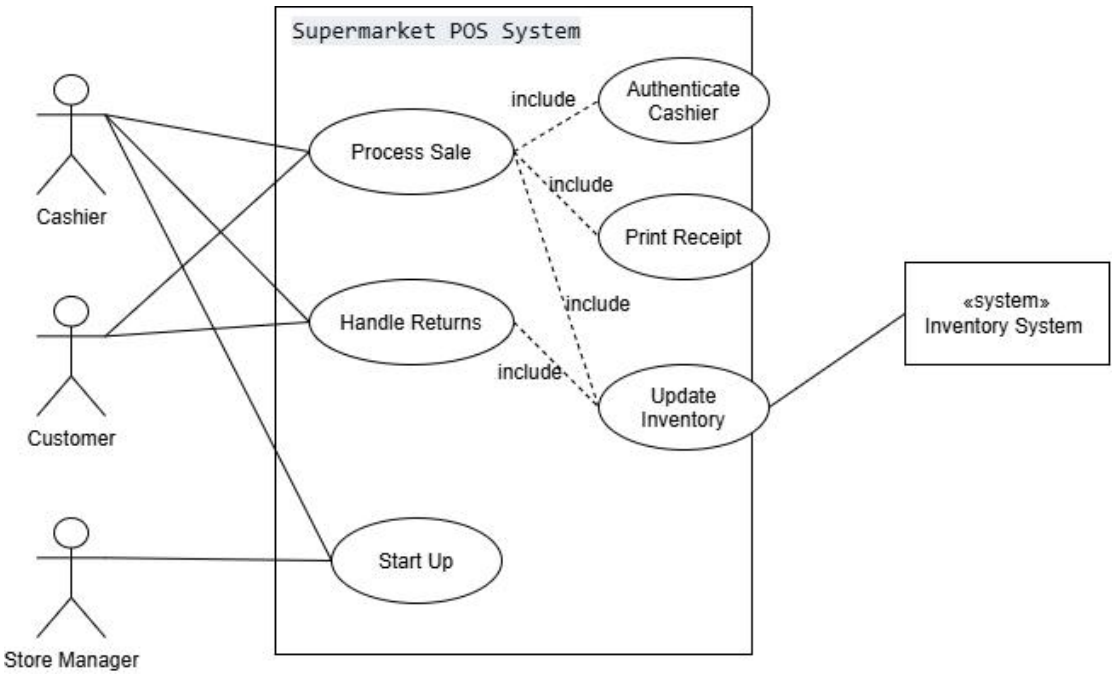

# Part 2 – The point-of-sale (POS) system

## Introduction

This part documents the system analysis and object-oriented design (OOA/D) of the POS system, following the iterative development approach. It includes the Use Case Model, Domain Model, Logical Architecture, and Object Design using UML diagrams.

## 1 Use Case Model

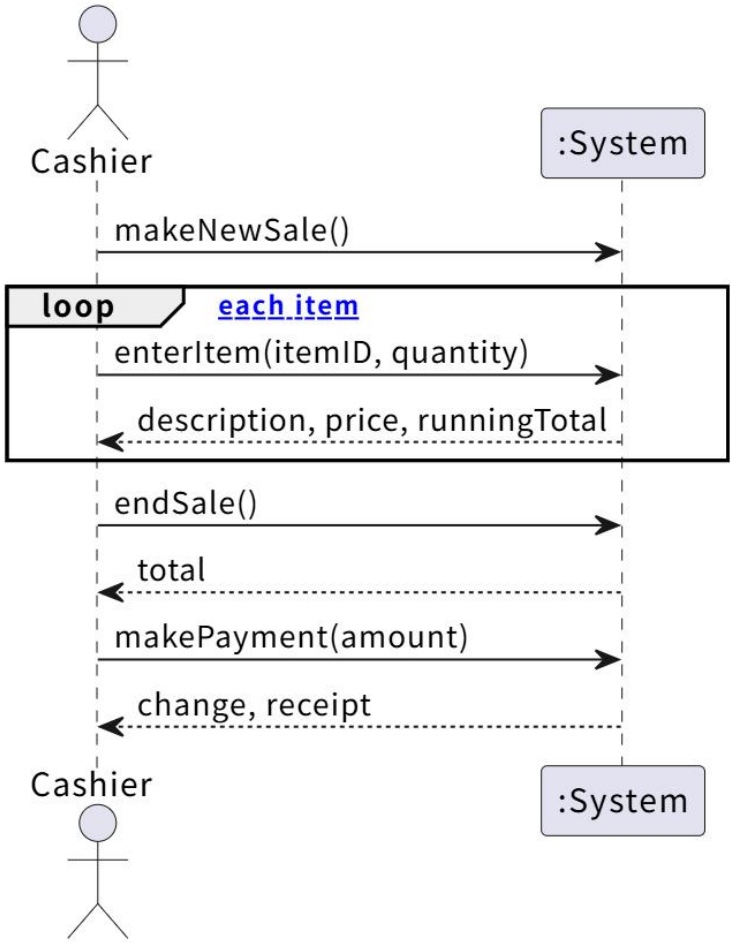
The use case model illustrates the system's functionality from the user's perspective.




The system boundary defines the POS application. The Cashier and Customer are the primary actors initiating the Process Sale and Handle Returns use cases. The Process Sale use case utilizes the <<include>> relationship to encapsulate mandatory steps such as authenticating the cashier, printing receipts, and updating the inventory system, ensuring modularity.

# 2 System Sequence Diagram (SSD)

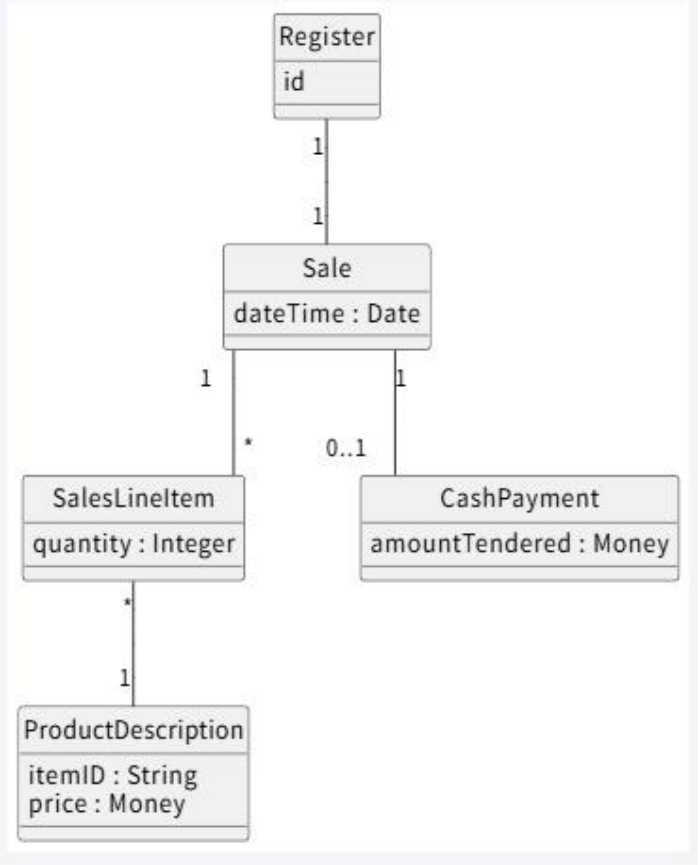
The SSD for the "Process Sale" main success scenario illustrates the system events generated by the Cashier.




The Cashier starts a new sale (makeNewSale), enters items in a loop (enterItem), ends the sale (endSale), and processes the payment (makePayment). The loop fragment indicates that multiple items can be entered before checking out.

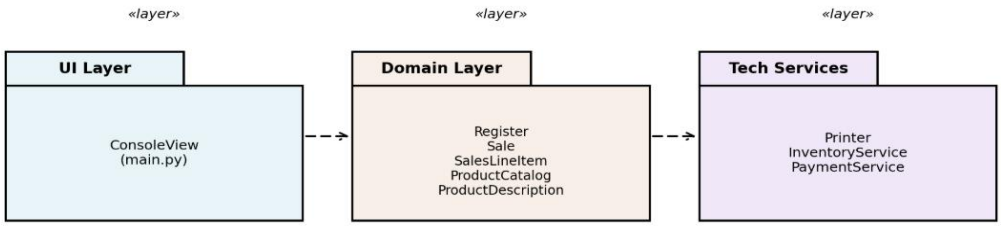
# 3 Domain Model

The domain model represents the conceptual classes and relationships in the POS domain. It ignores software-specific details to focus on real-world concepts.




# 4 Logical Architecture

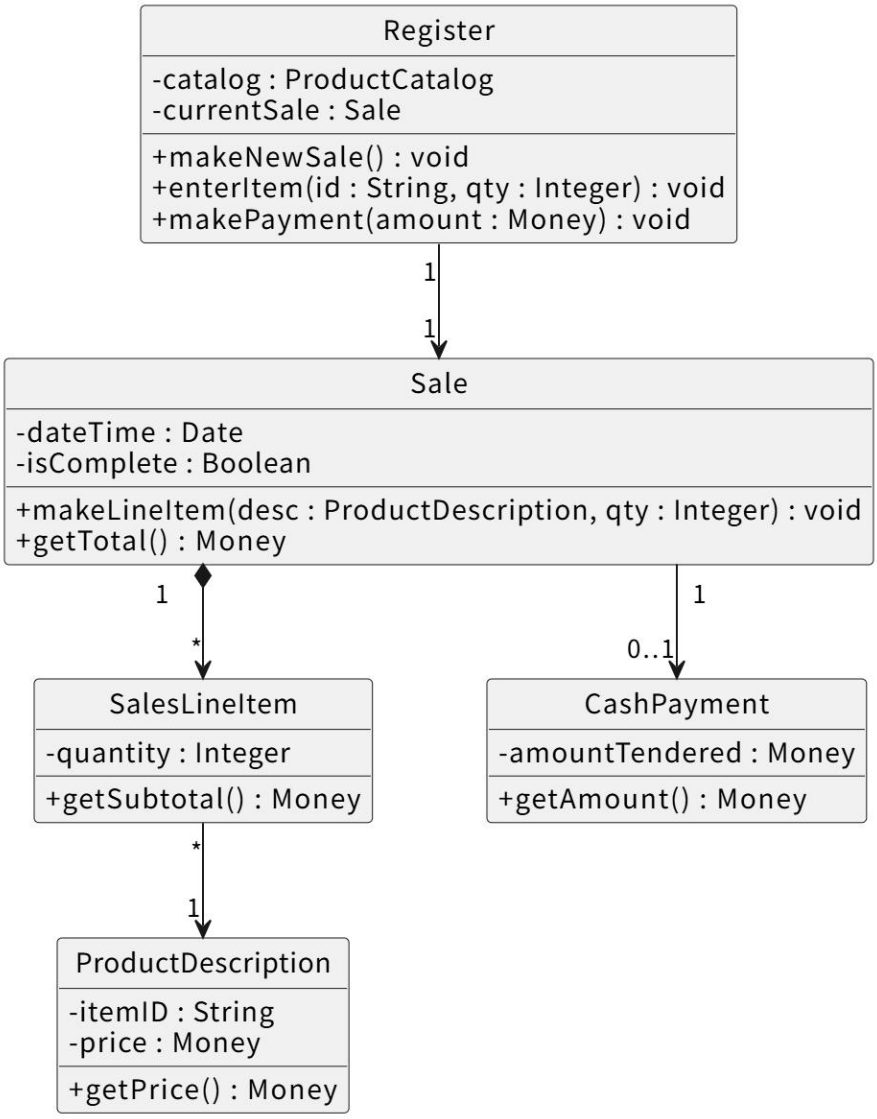
The POS system is designed using a layered architecture to ensure high cohesion and low coupling, separating user interfaces, core business logic, and technical services.




# 5 Object Design

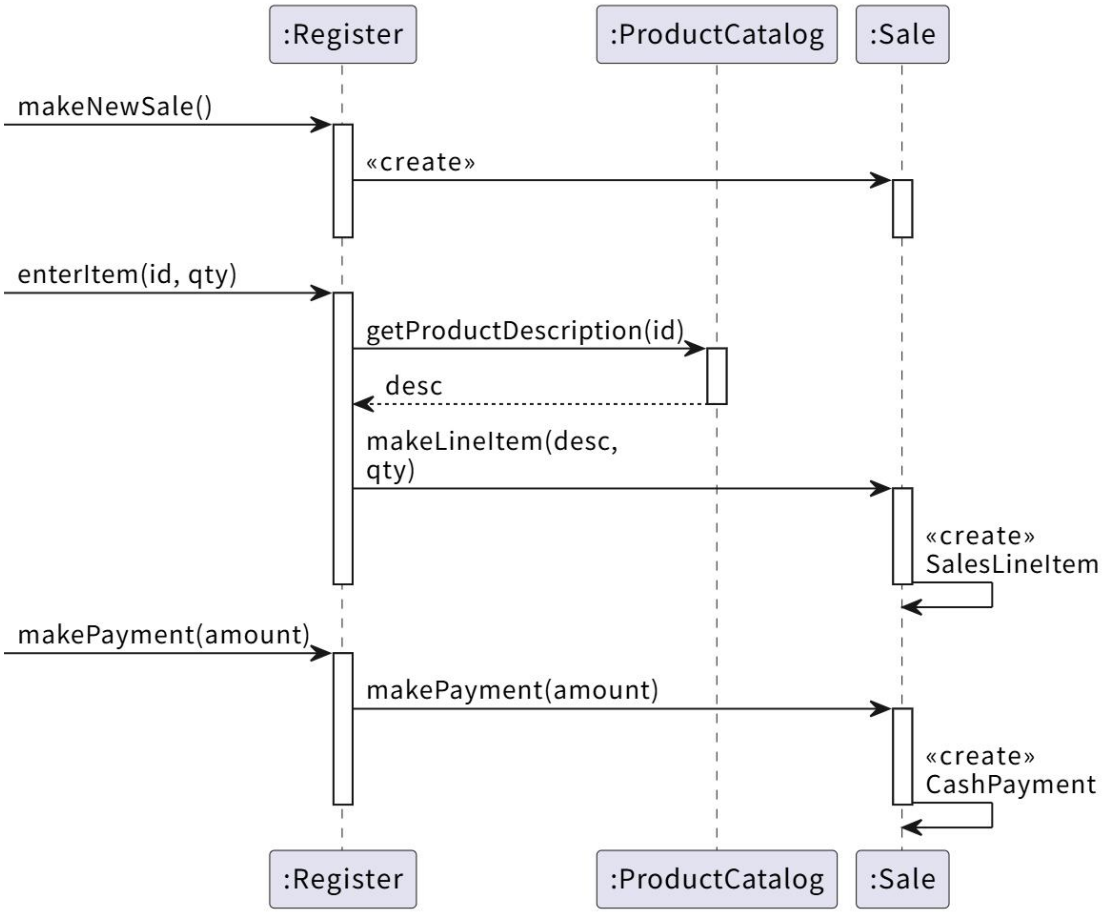
The object design translates the conceptual domain model into a software-specific design model.

**Design Class Diagram (DCD)**




The DCD defines the structural and behavioral specifications of the software classes. Visibility is explicitly marked (- for private attributes, + for public methods). For example, Register acts as a controller, while Sale maintains a collection of SalesLineItem objects (indicated by the composition relationship).

## Sequence Diagram for enterItem




This diagram applies the GRASP design patterns:

Controller: Register receives the enterItem message from the external actor.

Information Expert: Register sends a message to ProductCatalog to retrieve the product details because the catalog knows all products.

Creator: Register sends a makeLineItem message to Sale. The Sale then creates a new SalesLineItem instance.

# 6 Implementation Summary

The system was implemented in Python. The console output above demonstrates a complete sale scenario:

The cashier starts a new sale.

The cashier enters two Apples (ID: 1001) and one Bread (ID: 1002).

The system calculates the total ($8.00).

The customer pays $20.00, and the system calculates the correct change ($12.00)

